

FloodSim Phase 1: A Toy-Grid Rainfall and Surface Routing Simulator

FloodSim Project

May 3, 2026

Abstract

FloodSim Phase 1 is a deliberately simplified raster-grid flood simulator designed to stabilize semantics, tests, and exported outputs before the project expands into real-terrain ingestion and richer urban flood behavior. The model applies spatially uniform rainfall to each cell, computes local water-surface height as terrain elevation plus ponded depth, and routes a bounded fraction of water to lower orthogonal neighbors using a shared-grid snapshot. This paper documents the implemented Phase 1 contract, the rainfall and boundary assumptions, the step algorithm, and the main limitations that separate this prototype from a city-scale operational flood product.

1 Purpose and Scope

The long-term FloodSim product aims to support urban flood workflows for city planners, infrastructure teams, risk analysts, and insurers. Phase 1 does not attempt to solve that entire problem. Instead, it provides a trustworthy kernel with:

- explicit rainfall semantics
- explicit boundary semantics
- deterministic step behavior
- stable exported outputs
- lightweight automated tests

This phase should be read as a software and modeling contract, not as a validated hydrology or hydraulic model.

2 Model State Variables

The Phase 1 simulator operates on a regular raster grid. Each cell stores:

- terrain elevation $z_{r,c}$ in meters
- ponded water depth $h_{r,c}$ in meters
- surface height $s_{r,c} = z_{r,c} + h_{r,c}$

The current model uses only the four orthogonal neighbors of each cell. Diagonal flow is excluded in Phase 1.

3 Rainfall Contract

Rainfall is represented as a uniform intensity I in meters per hour. For a simulation step duration Δt in seconds, the rainfall depth injected into every cell during that step is:

$$\Delta h_{\text{rain}} = I \cdot \frac{\Delta t}{3600}$$

This is an intensity contract rather than a per-step depth contract. As a result:

- changing Δt changes the rainfall depth added in each step
- multiple short steps accumulate the same rainfall depth as one long step over the same total simulated duration
- rainfall is applied before any flow routing during the step

4 Routing Semantics

After rainfall is injected, the simulator examines each cell using a shared snapshot of the grid state. For cell (r, c) with water depth $h_{r,c}$, the algorithm:

1. computes the local surface height $s_{r,c}$
2. identifies all orthogonal neighbors with strictly lower surface height
3. computes a capped outflow:

$$q_{r,c} = h_{r,c} \cdot f_{\max}$$

where $f_{\max}$ is the configured maximum outflow fraction

4. splits that outflow among lower neighbors in proportion to relative surface drop
5. applies all accumulated cell updates only after the full grid scan completes

The snapshot rule matters: water received by a cell during one step cannot be relayed again until the next step.

5 Boundary Contract

Phase 1 exposes boundary handling explicitly in the simulation configuration, but only one mode is currently supported:

- Closed: water can move only to in-domain neighbors and cannot leave the raster across an outer edge

This is a deliberate simplification. Open edge outflow and more realistic domain handling are deferred to later phases.

6 Step Algorithm Pseudocode

1. Add uniform rainfall depth to every cell.
2. Allocate a zero-initialized delta array for all cells.
3. Scan the grid cell by cell.
4. Skip cells whose current water depth is zero.
5. Compute the source-cell surface height.
6. Collect orthogonal neighbors whose surface height is strictly lower.
7. If no lower neighbors exist, continue to the next cell.
8. Compute capped outflow as water depth multiplied by the maximum outflow fraction.
9. Subtract that outflow from the source-cell delta.
10. Split the outflow across lower neighbors in proportion to relative surface drop.
11. Add each neighbor share to the corresponding neighbor delta.
12. After the full scan completes, apply the delta array back to the grid.

7 Representative Figures

7.1 Orthogonal Routing Schematic

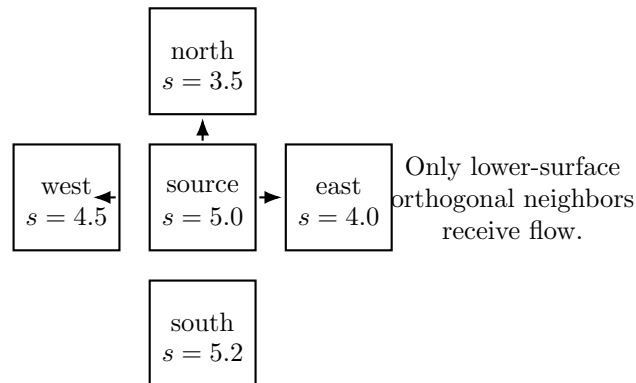


Figure 1: Phase 1 routes water only to lower orthogonal neighbors. The south neighbor is excluded because its surface is not lower than the source surface.

7.2 Rainfall Accumulation Graph

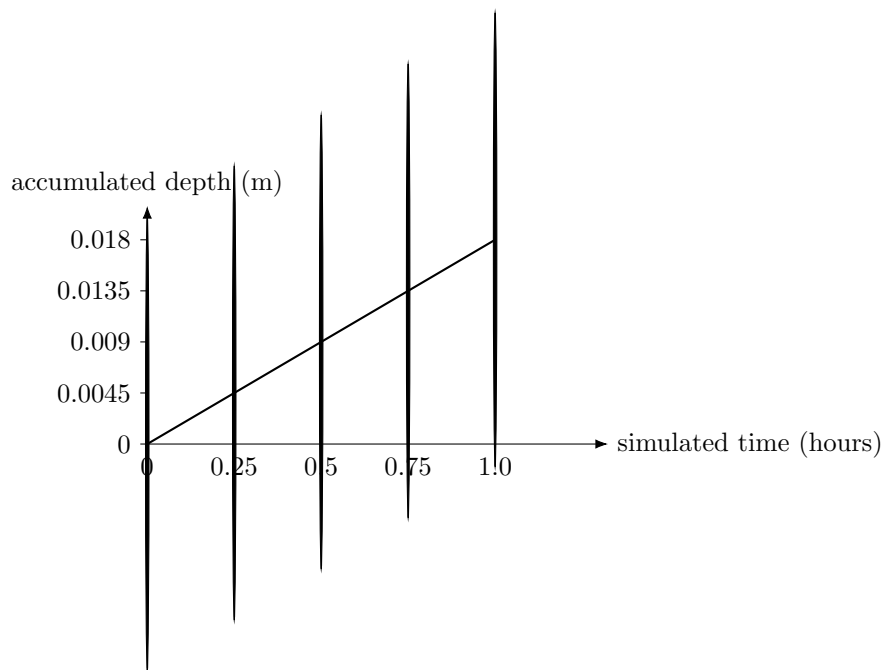


Figure 2: Example accumulation curve for a uniform rainfall intensity of 0.018 m/h. This is the same contract used by the Phase 1 example run.

8 Exported Outputs

The Phase 1 export contract is a text CSV with a metadata preamble:

```
floodsim`csv`version, 1
rows, row-count
cols, column-count
cell`size`m, cell-size-value
row, col, elevation`m, water`depth`m, surface`height`m
```

This keeps the output self-describing enough for scripts and future viewers while staying lightweight and easy to inspect.

9 Limitations and Non-Goals

Phase 1 deliberately does not include:

- shallow-water equations or physically rigorous hydrodynamics
- infiltration, evaporation, or subsurface processes
- storm sewer or drainage network modeling
- building blockage or urban roughness treatment
- open-boundary or coastal/tidal boundary handling
- calibration against observed flood events

These omissions are not accidents. They are explicit non-goals of the current phase.

10 Why Phase 1 Still Matters

Even with those limitations, this phase is valuable because it locks down:

- the meaning of rainfall inputs
- the routing contract
- the boundary contract
- the exported-output contract
- a tested baseline for later real-terrain and urban-surface work

That makes later phases easier to review and lowers the risk of building DEM import, viewers, and scenario tooling on top of ambiguous behavior.

11 Conclusion

FloodSim Phase 1 is best understood as a simulation-core prototype with explicit semantics rather than a finished urban flood product. Its role is to provide a small but stable foundation for subsequent phases: real-terrain ingestion, map-aligned outputs, richer rainfall scenarios, and urban flood behavior that can eventually support practical planning and risk workflows.